

```
In [31]: import pandas as pd
import numpy as np

df = pd.read_csv("accident_dataset.csv")
print("Dataset shape:", df.shape)
print("FIRST 10 ROWS OF THE DATASET:")
print(df.head(10))
```

Dataset shape: (132000, 30)

FIRST 10 ROWS OF THE DATASET:

	Country	Year	Month	Day of Week	Time of Day	Urban/Rural	Road Type	\
0	USA	2002	October	Tuesday	Evening	Rural	Street	
1	UK	2014	December	Saturday	Evening	Urban	Street	
2	USA	2012	July	Sunday	Afternoon	Urban	Highway	
3	UK	2017	May	Saturday	Evening	Urban	Main Road	
4	Canada	2002	July	Tuesday	Afternoon	Rural	Highway	
5	India	2010	May	Monday	Evening	Urban	Street	
6	China	2010	March	Monday	Afternoon	Rural	Street	
7	USA	2016	July	Friday	Afternoon	Rural	Main Road	
8	Japan	2014	August	Thursday	Afternoon	Rural	Highway	
9	USA	2007	April	Monday	Evening	Urban	Highway	

	Weather Conditions	Visibility Level	Number of Vehicles Involved	...	\
0	Windy	NaN	1	...	
1	Windy	168.311358	3	...	
2	Snowy	341.286506	4	...	
3	Clear	489.384536	2	...	
4	Rainy	348.344850	1	...	
5	Snowy	479.216834	2	...	
6	Foggy	386.176217	3	...	
7	Foggy	75.608688	3	...	
8	Rainy	387.828675	3	...	
9	Foggy	443.965408	3	...	

	Number of Fatalities	Emergency Response Time	Traffic Volume	\
0	2	58.625720	7412.752760	
1	1	58.041380	4458.628820	
2	4	NaN	9856.915064	
3	3	48.554014	4958.646267	
4	4	18.318250	3843.191463	
5	4	8.205994	360.951795	
6	3	17.851663	7607.804705	
7	2	46.740367	6061.407002	
8	2	NaN	3793.850542	
9	3	NaN	1140.429308	

	Road Condition	Accident Cause	Insurance Claims	Medical Cost	\
0	Wet	Weather	4	40499.856980	
1	Snow-covered	Mechanical Failure	3	6486.600073	
2	Wet	Speeding	4	29164.412980	
3	Icy	Distracted Driving	3	25797.212570	
4	Icy	Distracted Driving	8	15605.293920	
5	Dry	Speeding	7	NaN	
6	Wet	Weather	9	NaN	
7	Dry	Speeding	8	4262.755621	
8	Snow-covered	Mechanical Failure	5	37624.775980	
9	Snow-covered	Distracted Driving	0	15801.190080	

	Economic Loss	Region	Population Density
0	22072.878500	Europe	3866.273014
1	9534.399441	North America	2333.916224
2	58009.145120	South America	4408.889129
3	20907.151300	Australia	2810.822423
4	13584.060760	South America	3883.645634
5	45995.605250	South America	3626.074027
6	52342.431810	Asia	3408.182341
7	70652.223520	South America	408.296453
8	13724.630950	Europe	2058.898279

9	61948.862750	Australia	1840.206143
---	--------------	-----------	-------------

[10 rows x 30 columns]

```
In [32]: print("\n" + "="*80)
print("DATA TYPES OF ALL COLUMNS:")

print(df.dtypes)

print("Data Type Summary:")

print(df.info())
```

=====

DATA TYPES OF ALL COLUMNS:

Country	object
Year	int64
Month	object
Day of Week	object
Time of Day	object
Urban/Rural	object
Road Type	object
Weather Conditions	object
Visibility Level	float64
Number of Vehicles Involved	int64
Speed Limit	int64
Driver Age Group	object
Driver Gender	object
Driver Alcohol Level	float64
Driver Fatigue	int64
Vehicle Condition	object
Pedestrians Involved	int64
Cyclists Involved	int64
Accident Severity	object
Number of Injuries	int64
Number of Fatalities	int64
Emergency Response Time	float64
Traffic Volume	float64
Road Condition	object
Accident Cause	object
Insurance Claims	int64
Medical Cost	float64
Economic Loss	float64
Region	object
Population Density	float64

dtype: object

Data Type Summary:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 132000 entries, 0 to 131999

Data columns (total 30 columns):

#	Column	Non-Null Count	Dtype
0	Country	132000 non-null	object
1	Year	132000 non-null	int64
2	Month	132000 non-null	object
3	Day of Week	132000 non-null	object
4	Time of Day	132000 non-null	object
5	Urban/Rural	132000 non-null	object
6	Road Type	132000 non-null	object
7	Weather Conditions	132000 non-null	object
8	Visibility Level	125400 non-null	float64
9	Number of Vehicles Involved	132000 non-null	int64
10	Speed Limit	132000 non-null	int64
11	Driver Age Group	132000 non-null	object
12	Driver Gender	132000 non-null	object
13	Driver Alcohol Level	104893 non-null	float64
14	Driver Fatigue	132000 non-null	int64
15	Vehicle Condition	132000 non-null	object
16	Pedestrians Involved	132000 non-null	int64
17	Cyclists Involved	132000 non-null	int64
18	Accident Severity	132000 non-null	object
19	Number of Injuries	132000 non-null	int64
20	Number of Fatalities	132000 non-null	int64

21	Emergency Response Time	118824	non-null	float64
22	Traffic Volume	125400	non-null	float64
23	Road Condition	132000	non-null	object
24	Accident Cause	132000	non-null	object
25	Insurance Claims	132000	non-null	int64
26	Medical Cost	106979	non-null	float64
27	Economic Loss	118800	non-null	float64
28	Region	132000	non-null	object
29	Population Density	125400	non-null	float64

dtypes: float64(7), int64(9), object(14)
memory usage: 30.2+ MB
None

```
In [33]: print("NUMERICAL AND CATEGORICAL VARIABLES:")

numerical_vars = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_vars = df.select_dtypes(include=['object']).columns.tolist()

print(f"\nNumerical Variables ({len(numerical_vars)}):")
print("-" * 40)
for var in numerical_vars:
    print(f" • {var}")

print(f"\nCategorical Variables ({len(categorical_vars)}):")
print("-" * 40)
for var in categorical_vars:
    print(f" • {var}")

print(f"\nTotal Variables: {len(numerical_vars) + len(categorical_vars)}")
```

NUMERICAL AND CATEGORICAL VARIABLES:

Numerical Variables (16):

- Year
- Visibility Level
- Number of Vehicles Involved
- Speed Limit
- Driver Alcohol Level
- Driver Fatigue
- Pedestrians Involved
- Cyclists Involved
- Number of Injuries
- Number of Fatalities
- Emergency Response Time
- Traffic Volume
- Insurance Claims
- Medical Cost
- Economic Loss
- Population Density

Categorical Variables (14):

- Country
- Month
- Day of Week
- Time of Day
- Urban/Rural
- Road Type
- Weather Conditions
- Driver Age Group
- Driver Gender
- Vehicle Condition
- Accident Severity
- Road Condition
- Accident Cause
- Region

Total Variables: 30

1. Load the given road accident dataset and display the first 10 rows. List all numerical and categorical variables separately. Check the data types of all columns and interpret whether any column needs type conversion

Interpretation:

```
In [34]: #select numerical columns
numerical_cols = df.select_dtypes(include=['int64', 'float64'])

# Descriptive statistics
desc_stats = numerical_cols.describe()
```

```
print(desc_stats)

# Find variable with highest variation (Standard Deviation)
std_values = numerical_cols.std()

highest_variation = std_values.idxmax()
highest_std = std_values.max()

print("\nVariable with Highest Variation:", highest_variation)
print("Standard Deviation:", highest_std)
```

	Year	Visibility Level	Number of Vehicles Involved \
count	132000.000000	125400.000000	132000.000000
mean	2011.973348	275.104755	2.501227
std	7.198624	129.946180	1.117272
min	2000.000000	50.001928	1.000000
25%	2006.000000	162.422604	2.000000
50%	2012.000000	274.808977	3.000000
75%	2018.000000	388.070736	3.000000
max	2024.000000	499.999646	4.000000

	Speed Limit	Driver Alcohol Level	Driver Fatigue \
count	132000.000000	104893.000000	132000.000000
mean	74.544068	0.110463	0.500576
std	26.001448	0.066818	0.500002
min	30.000000	0.000002	0.000000
25%	52.000000	0.053893	0.000000
50%	74.000000	0.107667	1.000000
75%	97.000000	0.161436	1.000000
max	119.000000	0.249989	1.000000

	Pedestrians Involved	Cyclists Involved	Number of Injuries \
count	132000.000000	132000.000000	132000.000000
mean	1.000773	0.998356	9.508205
std	0.816304	0.817764	5.774366
min	0.000000	0.000000	0.000000
25%	0.000000	0.000000	5.000000
50%	1.000000	1.000000	9.000000
75%	2.000000	2.000000	15.000000
max	2.000000	2.000000	19.000000

	Number of Fatalities	Emergency Response Time	Traffic Volume \
count	132000.000000	118824.000000	125400.000000
mean	1.995439	32.486666	5039.695672
std	1.412974	15.900793	2861.300571
min	0.000000	5.000177	100.062626
25%	1.000000	18.694200	2554.836546
50%	2.000000	32.549514	5033.213155
75%	3.000000	46.288919	7523.931508
max	4.000000	59.999588	9999.997468

	Insurance Claims	Medical Cost	Economic Loss	Population Density
count	132000.000000	106979.000000	118800.000000	125400.000000
mean	4.495621	22999.326535	46028.388017	2505.985119
std	2.867347	13309.840366	26642.146034	1441.036753
min	0.000000	500.110091	1000.335085	10.002669
25%	2.000000	11594.699610	23253.528405	1257.505421
50%	4.000000	22687.634120	45484.381415	2504.196516
75%	7.000000	33869.677745	67731.981662	3757.099191
max	9.000000	49999.930130	99999.622970	4999.991745

Variable with Highest Variation: Economic Loss
Standard Deviation: 26642.1460344933

```
In [35]: import pandas as pd

# Load dataset
df = pd.read_csv("accident_dataset.csv")

# Mean and Median of Medical Cost
mean_medical_cost = df["Medical Cost"].mean()
```



```

median_medical_cost = df["Medical Cost"].median()

print("Mean Medical Cost:", mean_medical_cost)
print("Median Medical Cost:", median_medical_cost)

# Interpret skewness
if mean_medical_cost > median_medical_cost:
    print("Distribution is Positively (Right) Skewed.")
elif mean_medical_cost < median_medical_cost:
    print("Distribution is Negatively (Left) Skewed.")
else:
    print("Distribution is Approximately Symmetric.")

```

Mean Medical Cost: 22999.326534801297
Median Medical Cost: 22687.63412
Distribution is Positively (Right) Skewed.

2. a. Generate descriptive statistics for all numerical variables. Which variable shows the highest variation?

b. Compare the mean and median of Medical Cost. What does the difference indicate about Skewness?

Interpretation:

A.The variable with the highest variation is Economic Loss with a standard deviation of approximately 26,642.15. This indicates that the values of Economic Loss are spread much more widely around their mean compared to the other numerical variables. In practical terms, accident-related economic losses vary significantly from one accident to another. B.Mean Medical Cost = 22,999.33 Median Medical Cost = 22,687.63 Since the mean is slightly greater than the median, the Medical Cost distribution is positively (right) skewed. This means that a few accidents have very high medical costs, pulling the average upward. Most medical costs are concentrated around lower values, while some extreme values create a longer tail on the right side of the distribution.

```

In [36]: # Missing value count
missing_count = df.isnull().sum()

# Missing value percentage
missing_percent = (df.isnull().sum() / len(df)) * 100

# Create summary table
missing_summary = pd.DataFrame({
    "Missing Count": missing_count,
    "Missing Percentage": missing_percent
})

# Show only columns with missing values

```

```
missing_summary = missing_summary[missing_summary["Missing Count"] > 0]

print(missing_summary)
```

	Missing Count	Missing Percentage
Visibility Level	6600	5.000000
Driver Alcohol Level	27107	20.535606
Emergency Response Time	13176	9.981818
Traffic Volume	6600	5.000000
Medical Cost	25021	18.955303
Economic Loss	13200	10.000000
Population Density	6600	5.000000

```
In [37]: # Median Imputation

for col in ["Visibility Level", "Traffic Volume", "Population Density"]:
    df[col].fillna(df[col].median(), inplace=True)

df["Driver Alcohol Level"].fillna(
    df["Driver Alcohol Level"].median(),
    inplace=True
)
df["Emergency Response Time"].fillna(
    df["Emergency Response Time"].median(),
    inplace=True
)
df["Medical Cost"].fillna(
    df["Medical Cost"].median(),
    inplace=True
)
```

C:\Users\Sharon\AppData\Local\Temp\ipykernel_14120\889193006.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\Sharon\AppData\Local\Temp\ipykernel_14120\889193006.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\Sharon\AppData\Local\Temp\ipykernel_14120\889193006.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df[col].fillna(df[col].median(), inplace=True)
```

C:\Users\Sharon\AppData\Local\Temp\ipykernel_14120\889193006.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df["Driver Alcohol Level"].fillna(
```

C:\Users\Sharon\AppData\Local\Temp\ipykernel_14120\889193006.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.

The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method

od({col: value}, inplace=True)' or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
df["Emergency Response Time"].fillna(
```

C:\Users\Sharon\AppData\Local\Temp\ipykernel_14120\889193006.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing '`df[col].method(value, inplace=True)`', try using '`df.method({col: value}, inplace=True)`' or `df[col] = df[col].method(value)` instead, to perform the operation inplace on the original object.

```
df["Medical Cost"].fillna(
```

3.

a. Identify all variables containing missing values. Calculate the number and percentage of missing values in each column.

The missing value analysis identifies columns that contain incomplete observations. The count indicates the number of missing entries, while the percentage shows the proportion of missing data relative to the dataset size.

Columns with a high percentage of missing values require careful treatment, as they can affect the accuracy and reliability of statistical analysis and machine learning models. Appropriate imputation techniques should be applied before further analysis.

b. Explain and identify which missing value technique can be used for variables Visibility Level, Traffic Volume, and Population Density.

The variables **Visibility Level**, **Traffic Volume**, and **Population Density** are continuous numerical variables that may contain skewed distributions and outliers. Therefore, **median imputation** is the preferred technique because the median is resistant to extreme values and provides a more representative measure of central tendency than the mean.

Using median imputation helps preserve the overall distribution of the data while minimizing the impact of unusually high or low observations.

c. Explain and identify which missing value technique can be used for variables Driver Alcohol

Level, Emergency Response Time, and Medical Cost.

The variables **Driver Alcohol Level**, **Emergency Response Time**, and **Medical Cost** are numerical variables that may exhibit skewness and contain extreme values. Median imputation is recommended because it is robust to outliers and prevents distortion of the data distribution.

For Medical Cost and Emergency Response Time, a few unusually large values can significantly affect the mean. Replacing missing values with the median ensures a more stable and representative estimate.

d. How can missingness in Driver Alcohol Level be both MAR and MNAR? Explain with reference to the dataset.

Missing values in **Driver Alcohol Level** can arise from different mechanisms.

- **Missing At Random (MAR):** Missingness depends on other observed variables in the dataset, such as Accident Severity or Emergency Response Time. For example, severely injured drivers may not be tested for alcohol immediately, resulting in missing records.
- **Missing Not At Random (MNAR):** Missingness depends on the actual alcohol level itself. Drivers with high alcohol concentrations may refuse testing, or their results may be unavailable due to legal or reporting issues.

Therefore, Driver Alcohol Level may exhibit both MAR and MNAR characteristics depending on the underlying reason for the missing data.

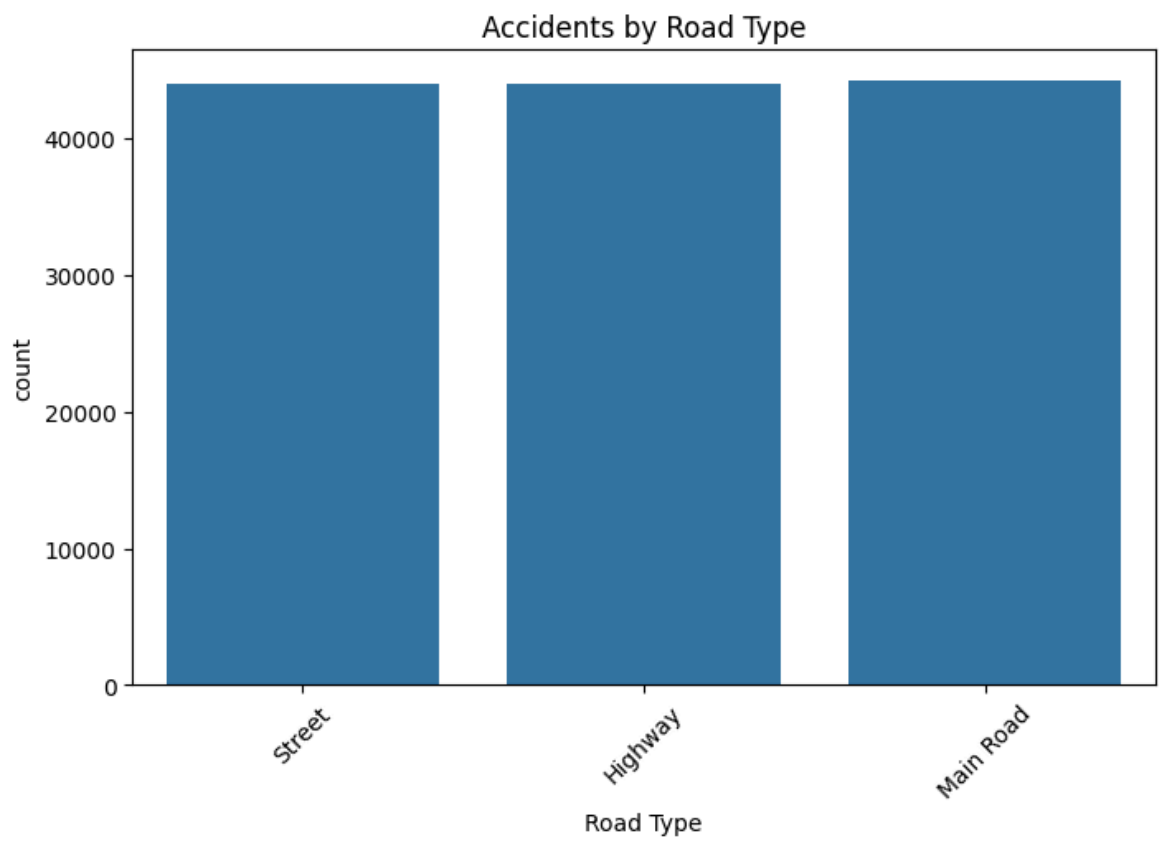
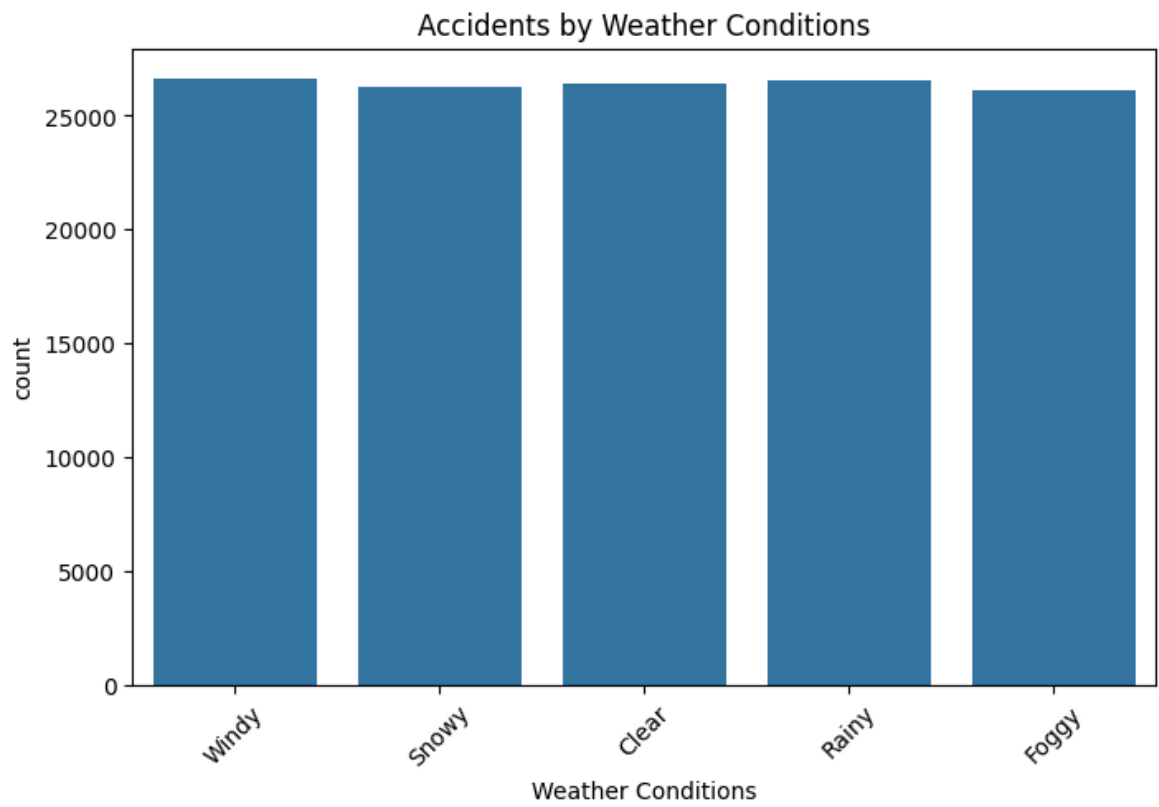
```
In [38]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

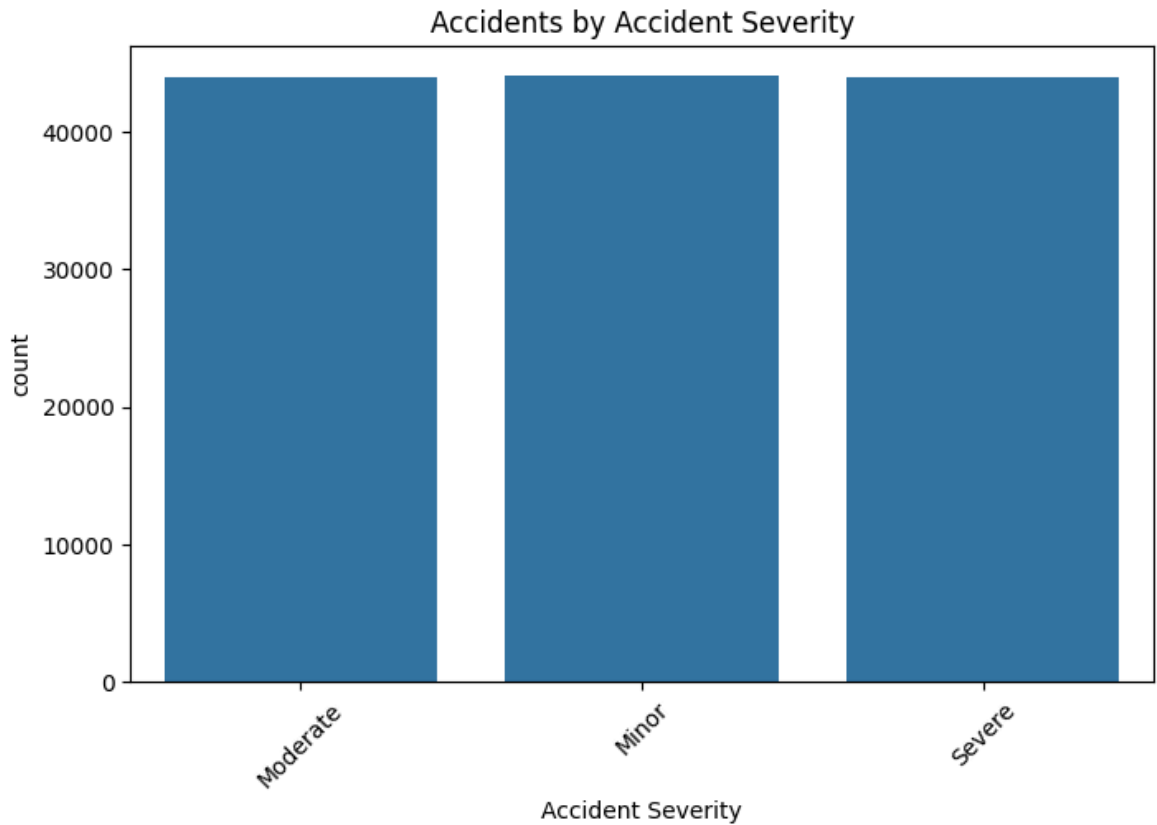
# Weather Conditions
plt.figure(figsize=(8,5))
sns.countplot(data=df, x='Weather Conditions')
plt.title('Accidents by Weather Conditions')
plt.xticks(rotation=45)
plt.show()

# Road Type
plt.figure(figsize=(8,5))
sns.countplot(data=df, x='Road Type')
plt.title('Accidents by Road Type')
plt.xticks(rotation=45)
plt.show()

# Accident Severity
plt.figure(figsize=(8,5))
sns.countplot(data=df, x='Accident Severity')
plt.title('Accidents by Accident Severity')
```

```
plt.xticks(rotation=45)  
plt.show()
```

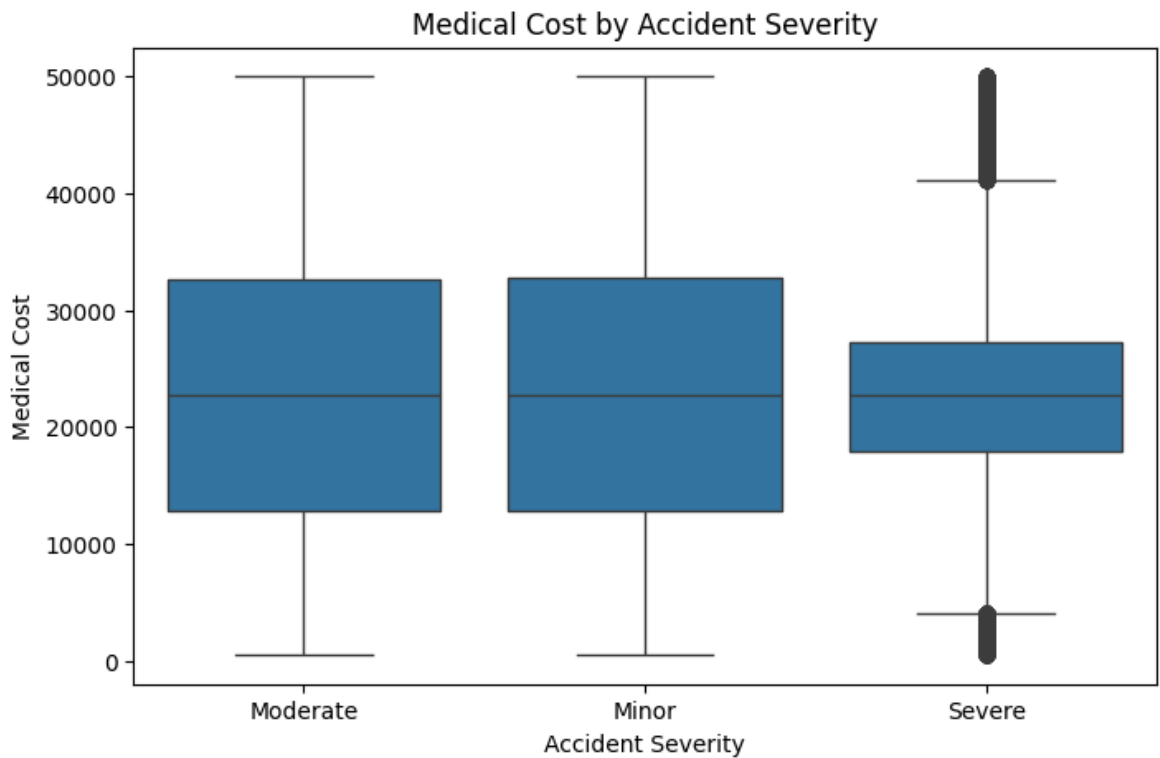
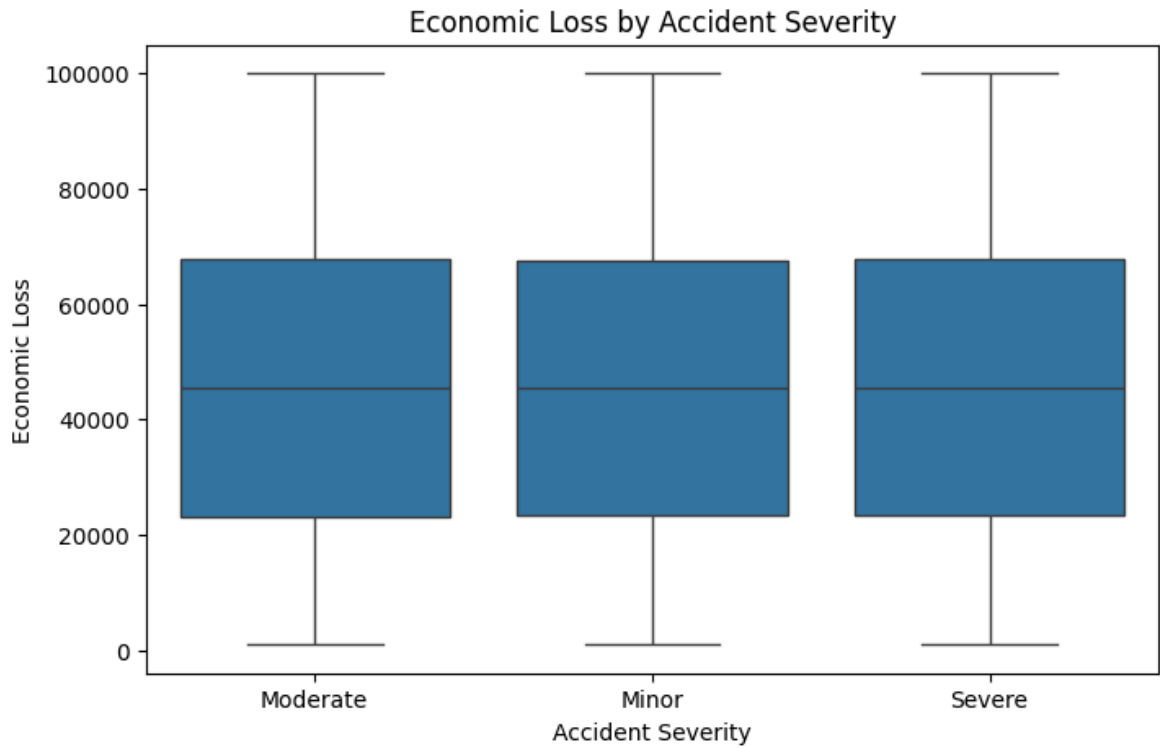




```
In [39]: import matplotlib.pyplot as plt
import seaborn as sns

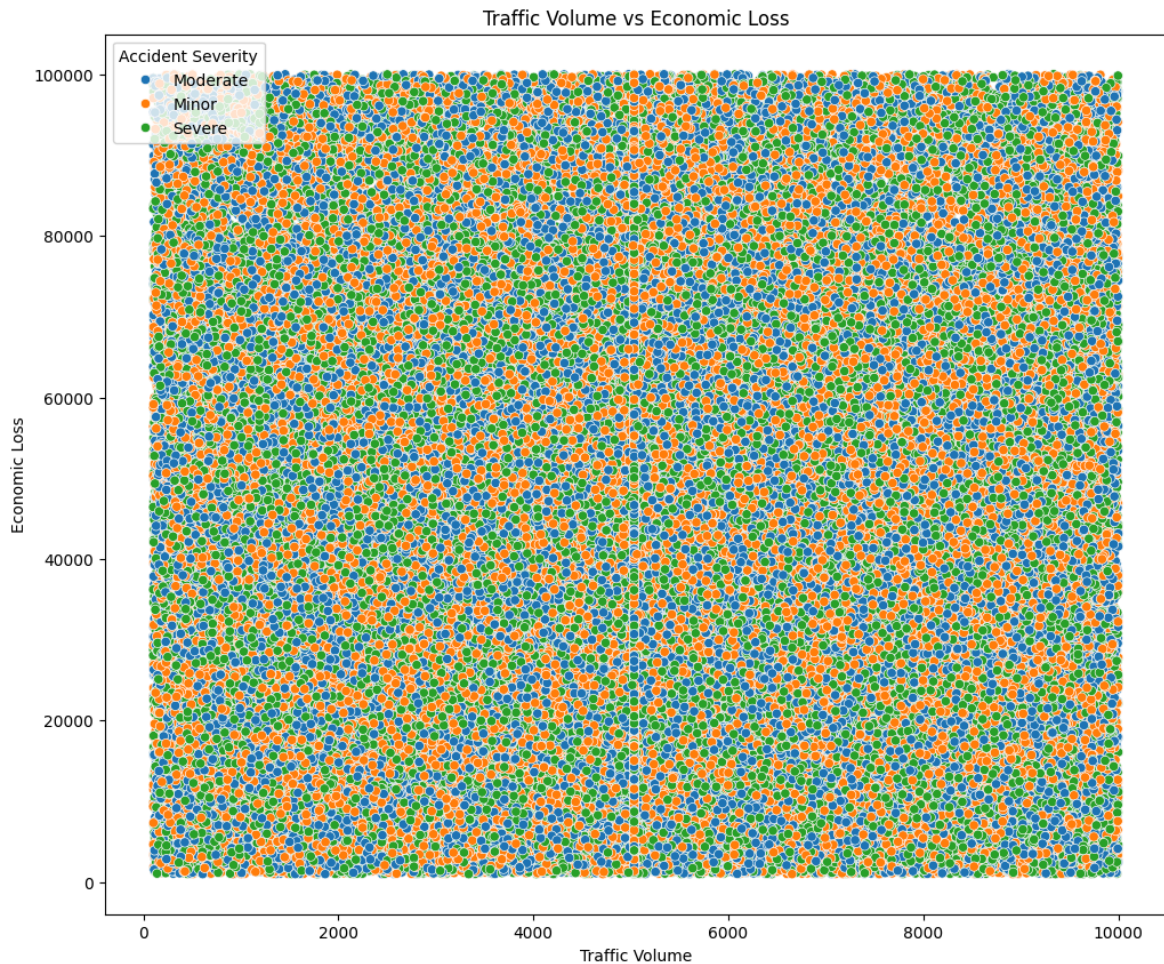
# Economic Loss vs Severity
plt.figure(figsize=(8,5))
sns.boxplot(
    data=df,
    x='Accident Severity',
    y='Economic Loss'
)
plt.title('Economic Loss by Accident Severity')
plt.show()

# Medical Cost vs Severity
plt.figure(figsize=(8,5))
sns.boxplot(
    data=df,
    x='Accident Severity',
    y='Medical Cost'
)
plt.title('Medical Cost by Accident Severity')
plt.show()
```



```
In [40]: plt.figure(figsize=(12, 10))
sns.scatterplot(
    data=df,
    x='Traffic Volume',
    y='Economic Loss',
    hue='Accident Severity'
)

plt.title('Traffic Volume vs Economic Loss')
plt.show()
```

4.

a. Plot a chart showing accidents by Weather Condition, Road Type and Accident Severity.

The Weather Condition chart shows how accident frequency varies across different weather situations. Clear weather is expected to have the highest number of accidents because it occurs most frequently. However, adverse weather conditions such as rain, fog, or snow may contribute to a higher accident risk due to reduced visibility and road friction.

This indicates that weather plays an important role in road safety and accident occurrence

b. Plot a boxplot for Economic Loss, Medical Cost grouped by Accident Severity.

The boxplot demonstrates that Economic Loss generally increases with Accident Severity. Severe accidents tend to have higher median losses and a wider spread of values compared to minor accidents.

The presence of outliers indicates that some severe accidents result in exceptionally high financial damage. This suggests a strong relationship between accident severity and economic impact.

c. Plot a scatter plot between Traffic Volume and Accident Severity / Economic Loss.

The scatter plot reveals the relationship between Traffic Volume and Economic Loss. As traffic volume increases, accidents may become more frequent and potentially lead to higher economic losses.

The clustering of severe accidents at higher traffic volumes suggests that congestion and increased vehicle interactions can contribute to greater accident severity and financial consequences.

```
In [41]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("accident_dataset.csv")

# Select numerical columns only
numerical_df = df.select_dtypes(include=['int64', 'float64'])

# Correlation matrix
corr_matrix = numerical_df.corr()

print(corr_matrix)

# Heatmap
plt.figure(figsize=(12,8))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title('Correlation Matrix of Numerical Variables')
plt.show()
```

	Year	Visibility Level \
Year	1.000000	0.002619
Visibility Level	0.002619	1.000000
Number of Vehicles Involved	-0.003528	0.000836
Speed Limit	-0.002744	-0.002101
Driver Alcohol Level	-0.000085	0.004446
Driver Fatigue	0.000545	0.002243
Pedestrians Involved	-0.002093	-0.004125
Cyclists Involved	-0.000077	-0.000087
Number of Injuries	0.001120	0.000446
Number of Fatalities	-0.004965	-0.001050
Emergency Response Time	-0.001644	-0.000987
Traffic Volume	-0.000355	-0.003266
Insurance Claims	0.001109	0.004576
Medical Cost	-0.007635	0.003036
Economic Loss	-0.004264	0.003401
Population Density	0.003848	-0.000321

	Number of Vehicles Involved	Speed Limit \
Year	-0.003528	-0.002744
Visibility Level	0.000836	-0.002101
Number of Vehicles Involved	1.000000	0.003759
Speed Limit	0.003759	1.000000
Driver Alcohol Level	0.007450	-0.001725
Driver Fatigue	0.001789	-0.001007
Pedestrians Involved	-0.004993	-0.001365
Cyclists Involved	-0.001884	0.001922
Number of Injuries	0.002234	-0.001118
Number of Fatalities	0.003353	0.000621
Emergency Response Time	0.002896	-0.000320
Traffic Volume	0.004056	-0.001019
Insurance Claims	-0.003621	0.000065
Medical Cost	-0.002732	0.000862
Economic Loss	-0.001200	0.000733
Population Density	-0.004649	0.001618

	Driver Alcohol Level	Driver Fatigue \
Year	-0.000085	0.000545
Visibility Level	0.004446	0.002243
Number of Vehicles Involved	0.007450	0.001789
Speed Limit	-0.001725	-0.001007
Driver Alcohol Level	1.000000	-0.002554
Driver Fatigue	-0.002554	1.000000
Pedestrians Involved	0.002990	-0.001152
Cyclists Involved	-0.000678	-0.001007
Number of Injuries	-0.004670	-0.004839
Number of Fatalities	0.002326	0.002685
Emergency Response Time	0.000784	0.001248
Traffic Volume	0.004581	-0.003752
Insurance Claims	-0.006424	0.002343
Medical Cost	-0.001417	-0.000378
Economic Loss	-0.002992	0.005159
Population Density	0.006029	0.003381

	Pedestrians Involved	Cyclists Involved \
Year	-0.002093	-0.000077
Visibility Level	-0.004125	-0.000087
Number of Vehicles Involved	-0.004993	-0.001884
Speed Limit	-0.001365	0.001922
Driver Alcohol Level	0.002990	-0.000678

Driver Fatigue	-0.001152	-0.001007
Pedestrians Involved	1.000000	0.002873
Cyclists Involved	0.002873	1.000000
Number of Injuries	-0.008377	0.000406
Number of Fatalities	0.003346	0.005160
Emergency Response Time	0.004931	-0.000824
Traffic Volume	0.001057	-0.001732
Insurance Claims	0.001225	0.000451
Medical Cost	-0.000449	-0.005555
Economic Loss	0.000481	-0.002048
Population Density	0.000868	0.004248

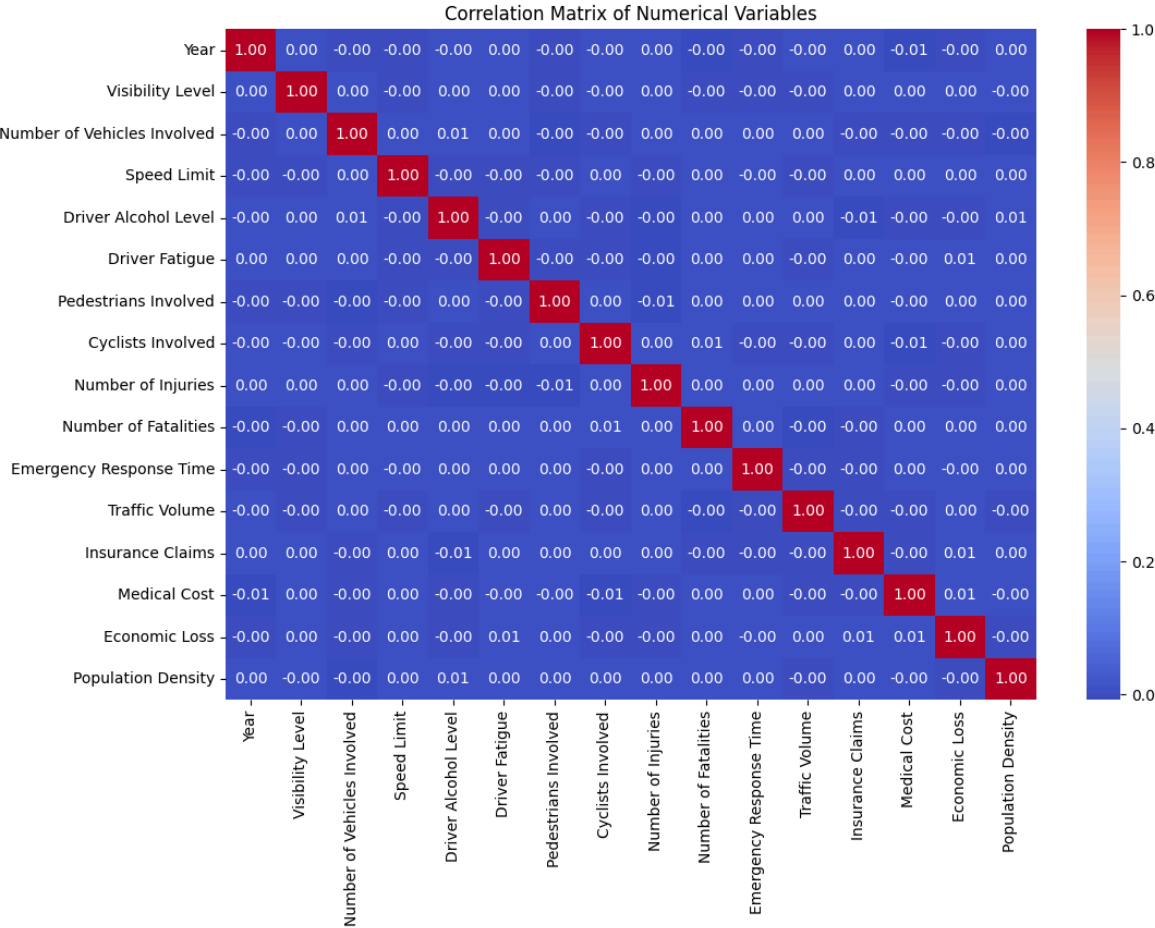
	Number of Injuries	Number of Fatalities \
Year	0.001120	-0.004965
Visibility Level	0.000446	-0.001050
Number of Vehicles Involved	0.002234	0.003353
Speed Limit	-0.001118	0.000621
Driver Alcohol Level	-0.004670	0.002326
Driver Fatigue	-0.004839	0.002685
Pedestrians Involved	-0.008377	0.003346
Cyclists Involved	0.000406	0.005160
Number of Injuries	1.000000	0.000389
Number of Fatalities	0.000389	1.000000
Emergency Response Time	0.000080	0.003269
Traffic Volume	0.001800	-0.004576
Insurance Claims	0.003242	-0.003330
Medical Cost	-0.002653	0.001379
Economic Loss	-0.002092	0.000927
Population Density	0.001859	0.004357

	Emergency Response Time	Traffic Volume \
Year	-0.001644	-0.000355
Visibility Level	-0.000987	-0.003266
Number of Vehicles Involved	0.002896	0.004056
Speed Limit	-0.000320	-0.001019
Driver Alcohol Level	0.000784	0.004581
Driver Fatigue	0.001248	-0.003752
Pedestrians Involved	0.004931	0.001057
Cyclists Involved	-0.000824	-0.001732
Number of Injuries	0.000080	0.001800
Number of Fatalities	0.003269	-0.004576
Emergency Response Time	1.000000	-0.001719
Traffic Volume	-0.001719	1.000000
Insurance Claims	-0.002718	-0.001470
Medical Cost	0.000265	-0.003406
Economic Loss	-0.000455	0.002142
Population Density	0.002279	-0.001675

	Insurance Claims	Medical Cost	Economic Loss \
Year	0.001109	-0.007635	-0.004264
Visibility Level	0.004576	0.003036	0.003401
Number of Vehicles Involved	-0.003621	-0.002732	-0.001200
Speed Limit	0.000065	0.000862	0.000733
Driver Alcohol Level	-0.006424	-0.001417	-0.002992
Driver Fatigue	0.002343	-0.000378	0.005159
Pedestrians Involved	0.001225	-0.000449	0.000481
Cyclists Involved	0.000451	-0.005555	-0.002048
Number of Injuries	0.003242	-0.002653	-0.002092
Number of Fatalities	-0.003330	0.001379	0.000927
Emergency Response Time	-0.002718	0.000265	-0.000455

Traffic Volume	-0.001470	-0.003406	0.002142
Insurance Claims	1.000000	-0.001837	0.006437
Medical Cost	-0.001837	1.000000	0.005073
Economic Loss	0.006437	0.005073	1.000000
Population Density	0.004102	-0.000283	-0.000975

	Population Density
Year	0.003848
Visibility Level	-0.000321
Number of Vehicles Involved	-0.004649
Speed Limit	0.001618
Driver Alcohol Level	0.006029
Driver Fatigue	0.003381
Pedestrians Involved	0.000868
Cyclists Involved	0.004248
Number of Injuries	0.001859
Number of Fatalities	0.004357
Emergency Response Time	0.002279
Traffic Volume	-0.001675
Insurance Claims	0.004102
Medical Cost	-0.000283
Economic Loss	-0.000975
Population Density	1.000000



5.Perform a correlation analysis on the numerical variables by creating a correlation matrix.

Identify the strength and direction of the correlations, and interpret the relationships between Emergency Response Time, Medical Cost, Economic Loss, Driver Alcohol Level, and Accident Severity.

Further, explain why categorical variables should not be directly included in correlation analysis without appropriate encoding.

Correlation Analysis Interpretation

The correlation matrix measures the strength and direction of linear relationships between numerical variables. Correlation values range from -1 to +1.

- Positive correlation indicates that both variables increase together.
- Negative correlation indicates that one variable increases while the other decreases.
- Values closer to ± 1 indicate stronger relationships, while values near 0 indicate weak or no linear relationship.

Relationship Between Key Variables

Emergency Response Time and Accident Severity

A positive correlation suggests that more severe accidents may require longer emergency response times due to increased complexity and medical attention requirements.

Medical Cost and Accident Severity

A positive correlation is expected because severe accidents typically result in greater injuries, hospitalization expenses, and treatment costs.

Economic Loss and Accident Severity

Economic Loss generally shows a positive correlation with Accident Severity since severe accidents lead to higher property damage, insurance claims, and societal costs.

Driver Alcohol Level and Accident Severity

A positive correlation indicates that higher alcohol levels may contribute to riskier driving behavior, increasing the likelihood of severe accidents.

Medical Cost and Economic Loss

These variables often exhibit a strong positive correlation because accidents involving expensive medical treatment usually generate higher overall economic losses.

Overall Conclusion

The correlation matrix helps identify variables that move together and provides insight into factors associated with accident severity and financial impact. Variables such as Medical Cost, Economic Loss, and Emergency Response Time are typically positively related to Accident Severity, indicating that more severe accidents have greater human and economic consequences.

Why Categorical Variables Cannot Be Directly Correlated

Correlation analysis is designed for numerical variables because it measures linear relationships using mathematical calculations involving means and variances.

Categorical variables such as:

- Country
- Road Type
- Weather Conditions
- Driver Gender
- Accident Cause

represent labels rather than numerical quantities. Directly assigning numbers to categories does not imply any meaningful order or distance between them.

For example:

Road Type:

- Highway = 1
- Urban Road = 2
- Rural Road = 3

The values 1, 2, and 3 are merely labels and do not indicate that Rural Road is three times greater than Highway. Therefore, computing Pearson correlation on such values would produce misleading results.

Appropriate Encoding Methods

Before correlation analysis, categorical variables should be transformed using encoding techniques such as:

1. One-Hot Encoding

- Creates binary columns for each category.

2. Label Encoding

- Suitable only for ordinal categories with a natural order.

3. Ordinal Encoding

- Used when categories have meaningful ranking.

After proper encoding, statistical analysis can be performed more appropriately depending on the nature of the variables.

```
In [42]: import pandas as pd

df = pd.read_csv("accident_dataset.csv")

columns = [
    'Medical Cost',
    'Economic Loss',
    'Traffic Volume',
    'Emergency Response Time'
]

for col in columns:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) |
                  (df[col] > upper_bound)]

    print(f"\n{col}")
    print(f"Number of Outliers: {len(outliers)}")
    print(f"Lower Bound: {lower_bound:.2f}")
    print(f"Upper Bound: {upper_bound:.2f}")
```

```
Medical Cost
Number of Outliers: 0
Lower Bound: -21817.77
Upper Bound: 67282.14
```

```
Economic Loss
Number of Outliers: 0
Lower Bound: -43464.15
Upper Bound: 134449.66
```

```
Traffic Volume
Number of Outliers: 0
Lower Bound: -4898.81
Upper Bound: 14977.57
```

```
Emergency Response Time
Number of Outliers: 0
Lower Bound: -22.70
Upper Bound: 87.68
```

```
In [43]: Q1 = df['Medical Cost'].quantile(0.25)
        Q3 = df['Medical Cost'].quantile(0.75)
```



```

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

medical_outliers = df[df['Medical Cost'] > upper]

print("Number of High Medical Cost Outliers:")
print(len(medical_outliers))

print("\nSample Outliers:")
print(
    medical_outliers[
        ['Medical Cost',
         'Accident Severity',
         'Number of Injuries',
         'Number of Fatalities']
    ].head()
)

```

Number of High Medical Cost Outliers:
0

Sample Outliers:
Empty DataFrame
Columns: [Medical Cost, Accident Severity, Number of Injuries, Number of Fatalities]
Index: []

6. a. Use the IQR method to identify outliers for Medical Cost, Economic Loss, Traffic Volume and Emergency Response Time.

The Interquartile Range (IQR) method was used to identify outliers in Medical Cost, Economic Loss, Traffic Volume, and Emergency Response Time.

The IQR method defines outliers as observations lying below:

Lower Bound = $Q1 - 1.5 \times IQR$

or above:

Upper Bound = $Q3 + 1.5 \times IQR$

The identified outliers represent unusually large or small values compared to the majority of observations.

Variables such as Medical Cost and Economic Loss are expected to contain more outliers because severe accidents can generate exceptionally high expenses and losses. Similarly, Emergency Response Time may contain outliers due to unusual delays in emergency services, while Traffic Volume may show outliers in highly congested areas.

b. Examine whether high Medical Cost values are errors or meaningful extreme cases.

High Medical Cost values should not automatically be treated as data errors.

To determine whether they are genuine observations, related variables such as Accident Severity, Number of Injuries, and Number of Fatalities should be examined.

If high Medical Cost values are associated with:

- Severe accidents
- Multiple injuries
- High fatality counts

then these observations represent meaningful extreme cases rather than incorrect data.

In accident datasets, extreme medical expenses are often expected because severe crashes require extensive medical treatment, hospitalization, surgeries, and long-term care.

Therefore, most high Medical Cost outliers are likely to be legitimate observations reflecting real-world accident severity rather than data entry errors.

7.

Accident Trends and Impact Metrics Over Time

Accident patterns vary across different years, months, days of the week, and times of the day.

- Peak accident frequencies are often observed during high-traffic periods such as morning and evening rush hours.
- Weekends and holidays may show different accident patterns due to recreational travel and increased alcohol consumption.
- Impact metrics such as Medical Cost, Economic Loss, and Number of Injuries tend to increase during periods with higher accident frequencies.
- Seasonal variations may also influence accident occurrence due to changing weather conditions and road visibility.

Overall, temporal analysis suggests that traffic density and human activity patterns significantly influence accident occurrence and associated costs.

Environmental and Road Condition Effects

Environmental factors play an important role in determining accident outcomes.

- Adverse weather conditions such as rain, fog, and storms reduce visibility and road friction, increasing accident risk.
- Poor visibility levels make it difficult for drivers to react quickly to hazards.
- Road conditions such as wet, damaged, or slippery surfaces contribute to loss of vehicle control.

- Certain road types, including highways and intersections, may experience more severe accidents due to higher vehicle speeds and traffic complexity.

The analysis indicates that unfavorable environmental and road conditions are associated with increased accident severity, injuries, and economic losses.

Factors Influencing Accident Severity and Impact

Several variables contribute significantly to accident severity and overall impact.

Key factors include:

- Driver Alcohol Level
- Traffic Volume
- Emergency Response Time
- Speed Limit
- Vehicle Condition
- Visibility Level
- Number of Vehicles Involved

Higher alcohol levels are associated with impaired driving behavior and greater accident severity.

Longer Emergency Response Times may increase injury severity and medical costs due to delayed treatment.

Accidents involving multiple vehicles typically result in higher Economic Loss and Medical Costs.

Traffic congestion and high-speed environments also contribute to more severe accident outcomes.

Therefore, both human behavior and environmental factors jointly influence accident severity and financial impact.

Geographic Variation in Accident Characteristics

Accident characteristics differ across countries, regions, and urban-rural locations.

- Urban areas generally experience a higher number of accidents because of greater traffic density.
- Rural areas may experience fewer accidents but often have higher severity due to higher travel speeds and delayed emergency response.
- Regions with higher Population Density tend to record more accidents because of increased vehicle and pedestrian interactions.
- Economic Loss and Medical Cost can vary significantly between regions due to differences in infrastructure, healthcare costs, and traffic management systems.

Geographic analysis highlights the importance of regional planning and targeted road safety policies to reduce accident frequency and severity.

